RansomRadar: An Intelligent ML Powered Early-Warning System

MINOR PROJECT REPORT

Submitted to

ORIENTAL UNIVERSITY, INDORE (M.P)

In partial fulfillment for the award of the degree
Of Bachelor of Engineering
In
Computer Science & Engineering

5th Semester | Session 2026–27

Supervised By:	**Submitted By:**
Suraj Batham Department of Computer Science & Engineering	Abhishek Yadav – PU00424EUG00647 Samarth Mishra – PU00424EUG00716 Pratik Goyal – PU00424EUG00707 Yash Chaudhary – PU00424EUG00735

Department of Computer Science & Engineering

ORIENTAL UNIVERSITY, INDORE (M.P)

CERTIFICATE

This is to certify that the Minor Project Report entitled “RansomRadar: An Intelligent ML Powered Early-Warning System” submitted by Abhishek Yadav (PU00424EUG00647), Samarth Mishra (PU00424EUG00716), Pratik Goyal (PU00424EUG00707) and Yash Chaudhary (PU00424EUG00735) is approved for the award of Degree of Bachelor of Engineering in Computer Science & Engineering.

This certificate is to be completed and authenticated by the authorized examiners of the Department of Computer Science & Engineering.

INTERNAL EXAMINER	**EXTERNAL EXAMINER**
Name: _____	Name: _____
Signature: _____	Signature: _____
DATE: _____	DATE: _____

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ORIENTAL UNIVERSITY, INDORE (M.P)

RECOMMENDATION

The Minor Project entitled "RansomRadar: An Intelligent ML Powered Early-Warning System" submitted by Abhishek Yadav (PU00424EUG00647), Samarth Mishra (PU00424EUG00716), Pratik Goyal (PU00424EUG00707) and Yash Chaudhary (PU00424EUG00735) is a satisfactory account of the bona fide work done under our supervision and is recommended towards partial fulfillment for the award of the Bachelor of Engineering (Computer Science & Engineering) degree by ORIENTAL UNIVERSITY, INDORE.

Supervisor: Suraj Batham

Signature: _____

Date: _____

Project Coordinator:	**Head of Department:**
Name: _____	Name: _____
Signature: _____	Signature: _____
Date: _____	Date: _____

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ORIENTAL UNIVERSITY, INDORE (M.P)

TABLE OF CONTENTS

S. No.	Particulars	Page No.
1.	INTRODUCTION	1
1.1	Purpose	1
1.2	Scope	1
1.3	Definition	1
1.4	Platform Specification	2
1.4.1	Hardware Specification	2
1.4.2	Software Specification	2
1.4.3	Implementation Language	2
2.	REQUIREMENT GATHERING AND ANALYSIS	3
2.1	System Analysis	3
2.1.1	Functional and Non-Functional Requirements	3
2.1.2	Existing System vs. Proposed System	3
2.2	Software Engineering Paradigm Applied	4
2.2.1	Description of Process Model	4
2.2.2	Advantages and Disadvantages	4
2.2.3	Reason for Selecting the Process Model	4
2.3	Requirement Analysis	5
2.3.1	Data Flow Diagram (DFD)	5
2.3.2	Identification of Actors	6
2.3.3	Identification of Use Cases	6
2.3.4	Use Case Diagram	7

TABLE OF CONTENTS (CONTINUED)

S. No.	Particulars	Page No.
2.3.5	Sequence Diagram	8
2.3.6	Class Diagram	9
2.3.7	Collaboration Diagram	10
2.3.8	Activity Diagram	11
3.	DESIGN	12
3.1	Design Classes	12
3.2	Entity Relationship (ER) Diagram	13
3.3	Database Tables	14
3.4	Snapshots	15
4.	IMPLEMENTATION	16
4.1	Implementation Details	16
4.2	Module Description	17
4.3	Algorithms / Methodology Used	18
5.	TESTING	20
5.1	Testing Strategy	20
5.2	Test Cases	21
5.3	Test Results	22
6.	CONCLUSION AND FUTURE SCOPE	23
6.1	Conclusion	23
6.2	Future Scope	23
7.	REFERENCES	24

1. INTRODUCTION

RansomRadar is an academic, behaviour-based early-warning prototype for ransomware-like activity in a designated folder. Instead of identifying malware solely through executable signatures, it records file events and examines behavioural and content changes that may indicate encryption or destructive operations. Its central aim is to warn an operator early and explain why the warning was raised. [1, 2]

1.1 Purpose

The purpose is to collect create, modify, move and delete events, turn them into measurable features, combine static rules with a learned normal profile, and show the evidence through a Streamlit dashboard. The project is intended for controlled demonstration and evaluation; it is not a replacement for endpoint protection or reliable offline backups.

1.2 Scope

The demonstrated scope is a recursively monitored sandbox on Windows 10/11, local SQLite storage, content sampling, canary alerts, burst rules, learned-envelope detection, an Isolation Forest score, dashboard inspection and CSV export. Monitoring is passive: the top-level detection path does not terminate processes or quarantine files. Setup creates decoys, while the simulator deliberately changes only demonstration files.

The archive also contains v2 research modules for richer streaming features, calibrated detection, reversible response and push notifications. These are described as component-level extensions because the supplied files do not establish a complete integrated v2 application. The project title, team and session follow the submitted synopsis rather than the alternative title suggested in the presentation guide.

1.3 Definition

Ransomware is malware that denies access to data, commonly by encryption, and demands payment. A canary is a decoy placed to reveal suspicious access. Entropy measures byte-distribution uncertainty; divergence means sampled content disagrees with an expected file type. An anomaly is a deviation from a learned normal profile, not proof of infection.

1.4 Platform Specification

1.4.1 Hardware Specification

The following is a practical laboratory recommendation, not a measured minimum or a claim about the authors' actual computer. No hardware inventory was supplied.

Component	Recommended laboratory configuration
Processor	Modern dual-core or better x64 processor
Memory	8 GB RAM recommended for dashboard, collector and training
Storage	At least 1 GB free for code, dependencies and demo records; more for experiments
Display	1366 × 768 or higher for dashboard inspection
Network	Not required for local monitoring; only for installation or opt-in remote alerts

1.4.2 Software Specification

Component	Software / role
Operating system	Windows 10/11 is the synopsis target; component review performed on Linux
Runtime	Python 3.13 in synopsis; README specifies 3.13+
Frontend	Streamlit ≥ 1.36
Event collector	watchdog ≥ 4.0
Learning and data	scikit-learn ≥ 1.5; pandas ≥ 2.0; NumPy ≥ 1.26
Persistence	SQLite through the Python standard library; joblib ≥ 1.3
Development tool	Visual Studio Code, as specified in synopsis

1.4.3 Implementation Language

Python provides filesystem-event handling, SQLite access, feature processing and model training in a common language. The top-level application uses mostly procedural modules with two main runtime classes, RadarHandler and LiveDetector. Newer rr/ modules use typed data classes and explicit interfaces. The v2 contract states Python 3.11+, whereas the synopsis states Python 3.13; deployment should use a tested version and freeze package versions before the final demonstration.

2. REQUIREMENT GATHERING AND ANALYSIS

2.1 System Analysis

Requirements were reconstructed from the supplied synopsis, source, integration contract and evaluation records. No interview transcripts were supplied. The operator needs early warnings with evidence that helps distinguish suspicious activity from legitimate changes.

2.1.1 Functional and Non-Functional Requirements

ID	Requirement	Classification
FR1	Observe recursive create, modify, move and delete events.	Functional
FR2	Save timestamps, paths and sampled forensic attributes.	Functional
FR3	Evaluate canary, extension-change, deletion, note and burst rules.	Functional
FR4	Train normal envelope; calculate an Isolation Forest display score.	Functional
FR5	Store explained rule, ML or fused alerts and display evidence.	Functional
FR6	Export alert and experiment data as CSV.	Functional
NFR1	Remain passive in the demonstrated top-level detection path.	Safety
NFR2	Limit content sampling to 4096 bytes and throttle repeated reads.	Efficiency
NFR3	Continue collecting when individual sample or notification operations fail.	Reliability
NFR4	Keep local data local unless an optional webhook is explicitly configured.	Privacy
NFR5	Explain alerts in readable language; evaluate false alarms as well as detection.	Usability / quality

2.1.2 Existing System vs. Proposed System

RansomRadar combines file behaviour, decoys, content checks and a learned envelope, rather than relying only on signatures or entropy. This does not imply commercial antivirus lacks behavioural detection. Legitimate bulk activity can trigger static rules; workload coverage limits the content checks and baseline. [2, 3]

2.2 Software Engineering Paradigm Applied

2.2.1 Description of Process Model

An iterative and incremental process is appropriate for this project. The supplied archive does not include a development diary proving a formal process was followed; the model below is the engineering interpretation used to organize the implementation and its next validation cycle.

Iteration	Work product	Verification gate
1. Collection	Sandbox, decoys, watcher, local event schema	Known file operations produce correct stored events
2. Rules	Burst thresholds and direct canary checks	Boundary cases and benign workload checks
3. Learning	Features, training envelope, Isolation Forest score	Separate normal training and evaluation sessions
4. User evidence	Dashboard, CSV and notifications	Alert explanation agrees with stored evidence
5. Evaluation	Fast, stealth and partial-rewrite simulations	Report delays and false positives honestly
6. v2 integration	Multi-scale features, calibration and guarded response	Resolve missing imports; full pytest; end-to-end regression

2.2.2 Advantages and Disadvantages

Advantages include early delivery of a usable collector, small testable modules, rapid adjustment of thresholds, and an explicit separation between the passive prototype and experimental response code. Each increment can be demonstrated independently. Disadvantages include compatibility work between frozen legacy modules and new schemas, dependency drift, duplicated functionality, and the risk that an impressive standalone component is mistaken for an integrated product.

2.2.3 Reason for Selecting the Process Model

Ransomware-like behaviour includes both fast bursts and slow rewrites, so initial assumptions need feedback from controlled tests. Incremental development lets the team first verify event correctness, then rule behaviour, then training and explanation. A final integration gate is necessary: the supplied archive contains missing v2 simulation and evaluation modules, making a purely feature-completion view of progress misleading.

2.3 Requirement Analysis

2.3.1 Data Flow Diagram (DFD)

The level-1 DFD follows the actual top-level program flow. The event store is the boundary between collection and later feature evaluation. Rules and the learned model are parallel evidence sources, not sequential replacements.

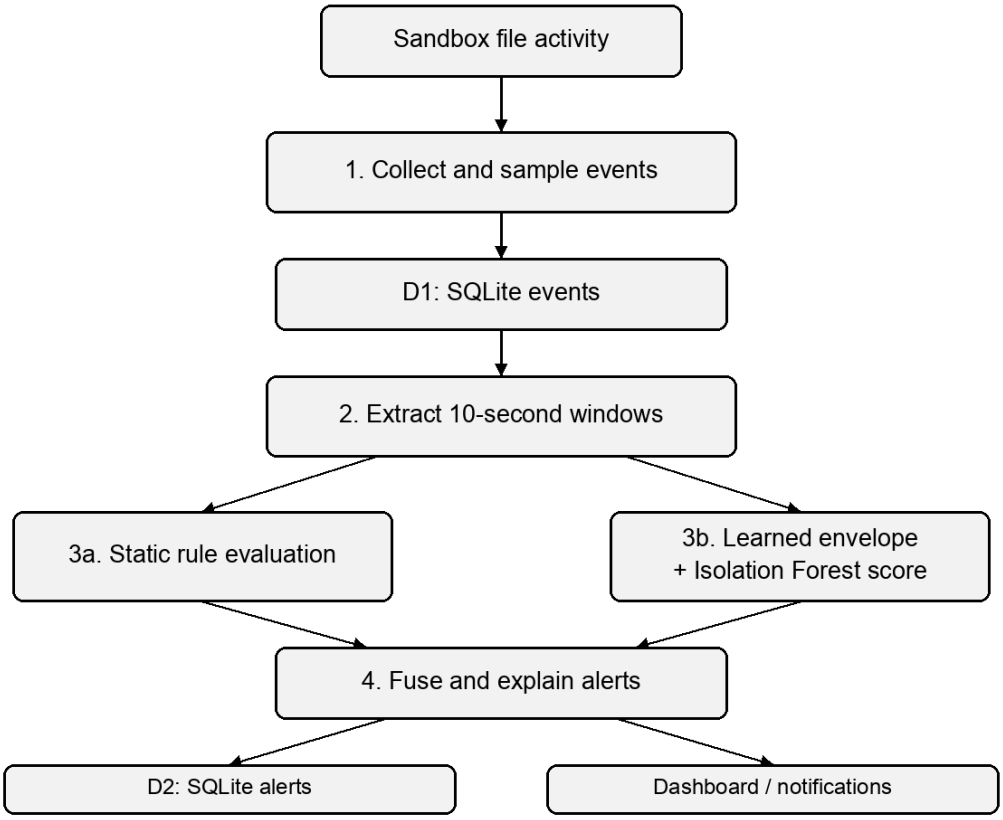


Figure 2.1: Level-1 data flow of the demonstrated passive prototype.

The collector persists file metadata and a small content sample's derived measurements. The detector reads records, groups them into fixed buckets and stores an explained verdict. The dashboard reads the database. The model bundle is stored separately in data/model.joblib; no raw sample bytes are part of the declared event schema.

2.3.2 Identification of Actors

Actor	Responsibilities
Operator / student	Starts monitoring, creates training data, trains, inspects evidence, exports CSV and stops services.
Filesystem / watchdog	External source of file-event callbacks; may emit duplicate or delayed events.
Controlled simulator	Generates benign, bulk-copy and ransomware-like operations inside the sandbox.
Notification destination	Windows desktop or an optional webhook receiver; not required for local detection.
Faculty reviewer	Assesses demonstration, report and limitations; has no special runtime privileges.

2.3.3 Identification of Use Cases

Use case	Precondition	Main outcome
UC1 Monitor sandbox	Dependencies installed; sandbox exists	Events saved; detector thread active
UC2 Learn normal activity	Representative normal records available	Envelope, forest and profile persisted
UC3 Inspect warning	At least one alert stored	Severity, explanation and feature evidence visible
UC4 Export records	Stored records available	CSV downloaded from dashboard
UC5 Evaluate scenarios	Controlled sandbox; collector off during reset	Comparable session-detector records produced
UC6 Stop safely	Collector running	Observer stopped and joined

UC3 alternate flow: an unreadable or already deleted file has missing forensic values, so the operator must not interpret absent entropy as proof that the file was safe. UC5 requires recording normal workloads too; successful detection without false-alarm measurement is incomplete evaluation. Role-based access control and user authentication are not implemented in the supplied local dashboard.

2.3.4 Use Case Diagram

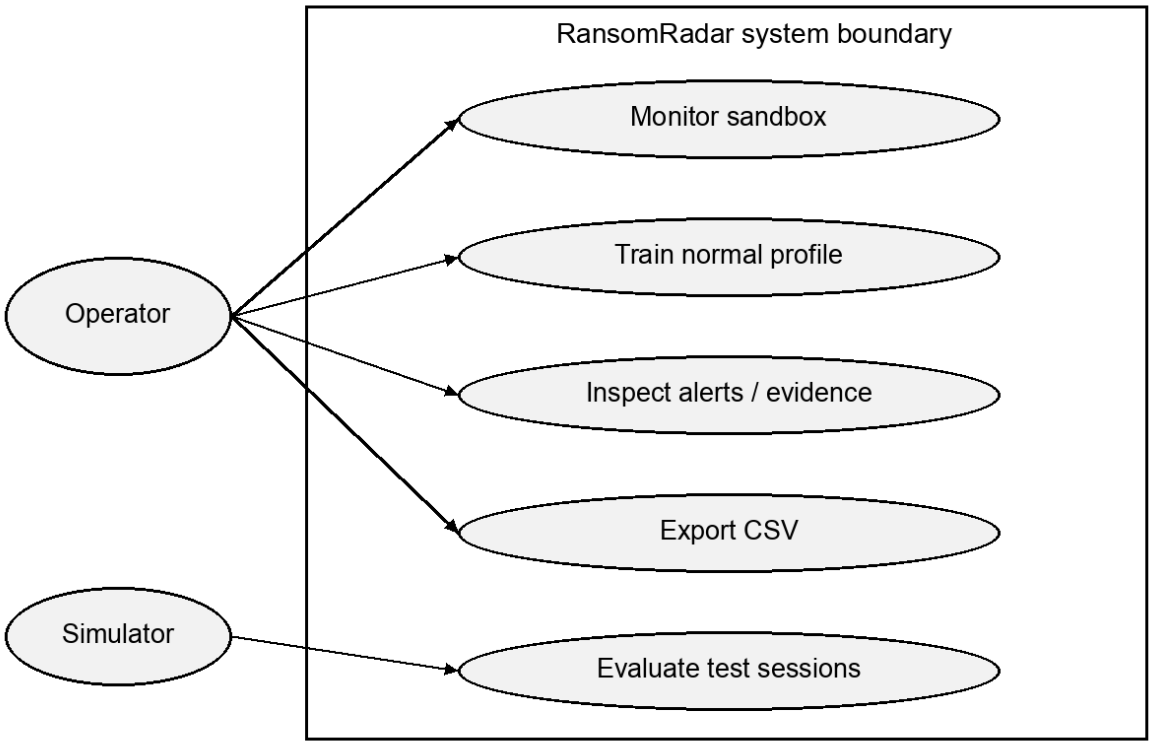


Figure 2.2: Operator and simulator interactions with the passive system.

The use case boundary intentionally excludes automatic process termination, kernel collection and recovery of files. Those functions are either future work for the passive synopsis or standalone rr/ extensions not wired into app.py and collector.py. An operator can clear stored events from the dashboard, so that control must be used with care when preserving evaluation evidence.

Training is distinct from evaluation: the operator should collect a range of legitimate edits and bulk operations before fitting the envelope. The dashboard describes detection as rules-only if there is no model bundle. Once training is available, the live detector can load it and supply both learned and static evidence.

2.3.5 Sequence Diagram

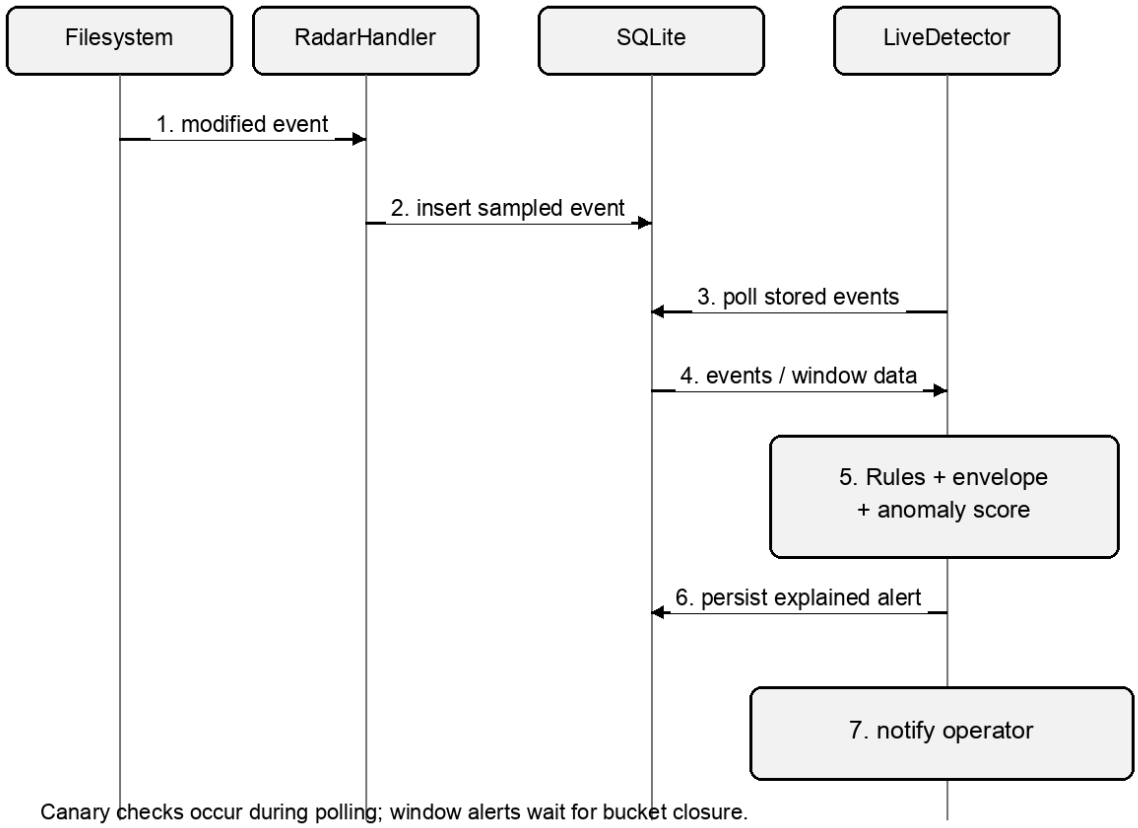


Figure 2.3: Typical modified-file event followed by polling and alert persistence.

RadarHandler receives the watchdog callback, samples modified files when allowed by the per-path throttle, and writes a normalized event record through db.py. LiveDetector polls at a configured three-second interval and checks recent canary paths before processing closed time buckets.

For ordinary window alerts, the detector waits until the bucket end plus a two-second grace interval, then computes rules and learned-envelope violations. This is why “instant” canary warnings and window-based detection have different clocks. The archived harness assigns a canary detection timestamp to the event itself; it does not measure the desktop notification’s wall-clock latency.

2.3.6 Class Diagram

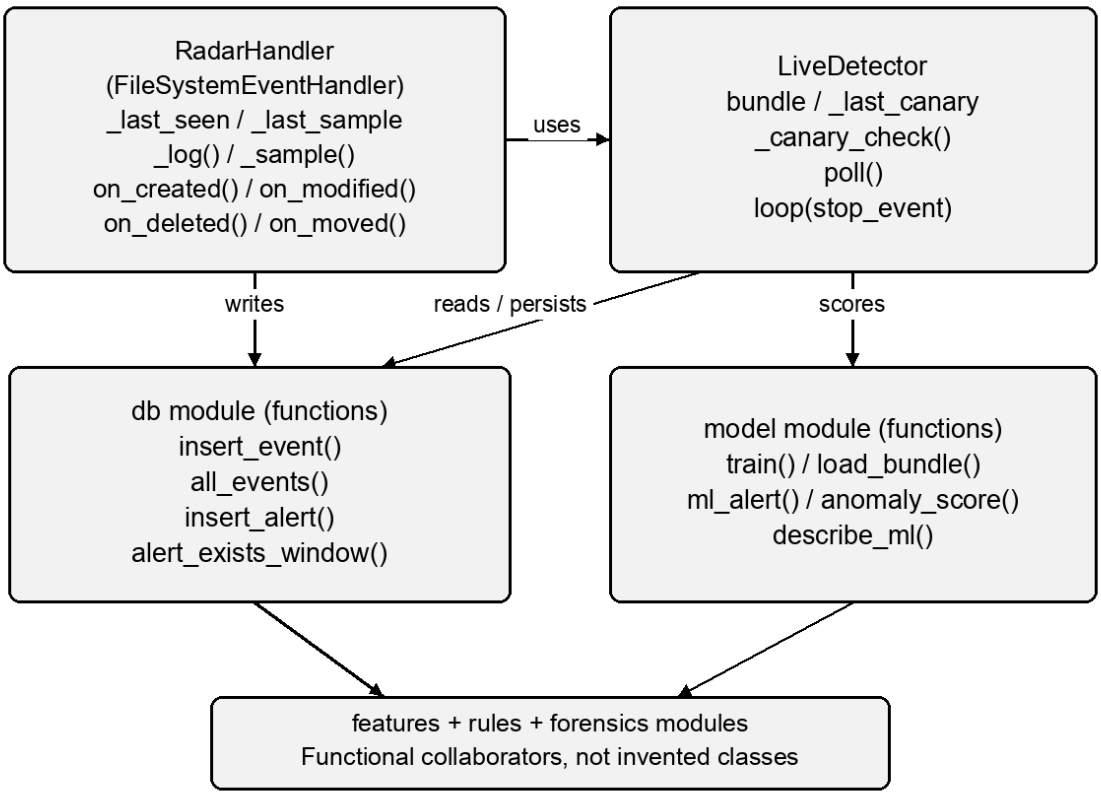


Figure 2.4: Runtime classes and functional collaborators in the top-level prototype.

The diagram preserves the actual implementation style: db, model, features, rules and forensics are modules exposing functions, not fabricated object-oriented classes. RadarHandler inherits FileSystemEventHandler and keeps duplicate-event and resampling state. LiveDetector maintains a loaded model bundle and canary cooldown state.

The rr/ extension defines additional typed entities, including NormalisedEvent, Window, Detector, ShadowJournal, ResponsePolicy, ResponseLadder and Notifier. They have distinct responsibility boundaries, but the provided top-level app does not instantiate these extension objects. Section 3.1 documents them without treating them as live integrations.

2.3.7 Collaboration Diagram

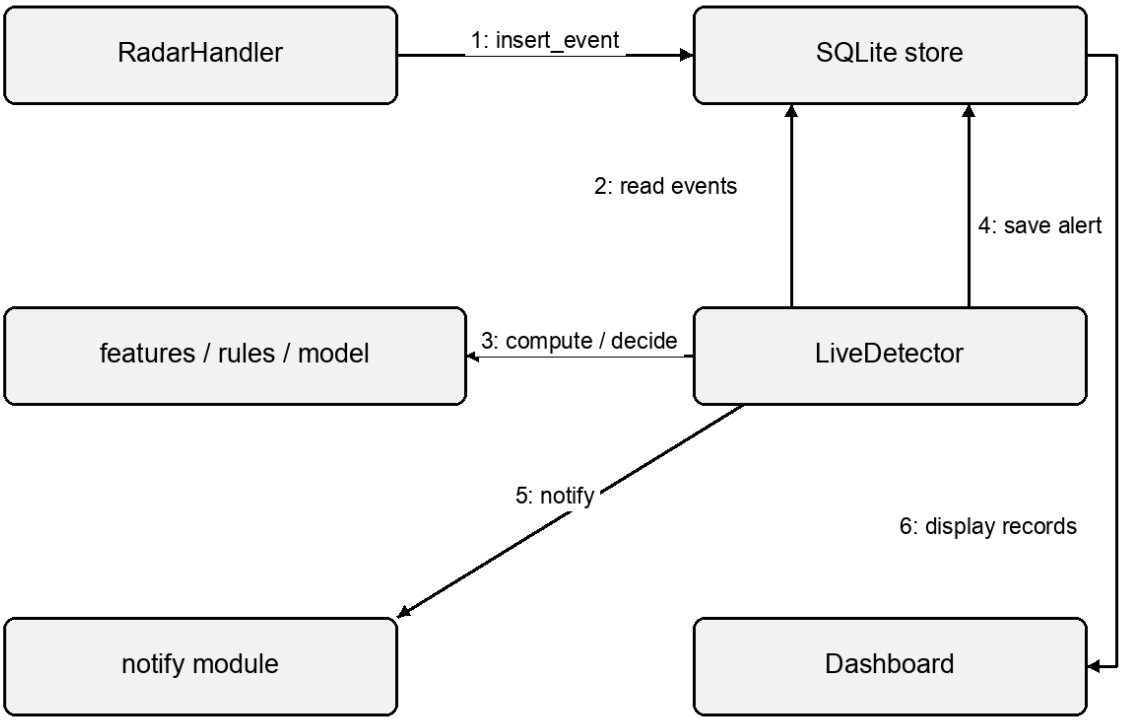


Figure 2.5: Numbered object/module collaboration for collection and warning.

Messages 1–4 describe the evidence path: persist a sampled event, read candidate activity, compute the feature verdict, and write an alert. Message 5 delegates notification, while the dashboard independently reads stored records for message 6. The display is not the authoritative detector; its refresh cycle does not control alert generation.

All persistence occurs through SQLite, so the collector and dashboard can run in separate processes. db.init_db enables write-ahead logging and a busy timeout. These settings support concurrent access in a small prototype but do not prove enterprise throughput. The archived database and experiment CSV are separate artifacts and should not be assumed to describe one identical run.

2.3.8 Activity Diagram

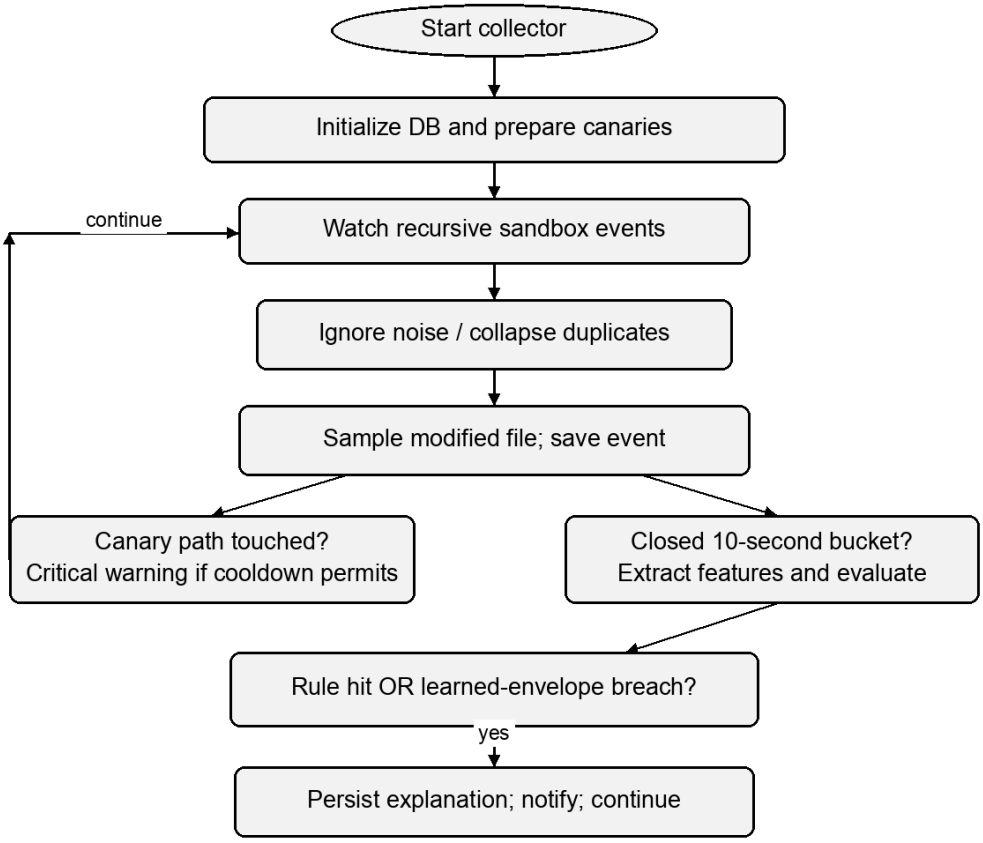


Figure 2.6: Monitoring, direct canary checking and closed-bucket evaluation.

The observer repeats until the operator stops the service. A missing forensic sample does not suppress event collection. Duplicate and noise filters reduce unnecessary writes but can also discard legitimate evidence, so filter policy should be tested against real workloads. No-hit evaluation continues without creating an alert.

3. DESIGN

3.1 Design Classes

Entity / module	Design responsibility	Status in supplied archive
RadarHandler	Callback handling, filtering, duplicate collapse and sampled forensics	Used by legacy collector
LiveDetector	Periodic evaluation, canary check, fusion and persistence	Used by legacy collector
NormalisedEvent / Window	Typed events and streaming window representation	rr/core source present
Detector	ECOD-style + Isolation Forest ensemble; calibration and sequential decisions	rr/detect source present; not legacy runtime
ShadowJournal	Root-confined, capped snapshots and byte-verified restoration	Source present; component tests exercised
ResponsePolicy / ResponseLadder	Dry-run escalation, limits, human gates and audit trail	Source present; component tests exercised
Notifier	Dry-run push notification with transport abstraction	Source present; fake-transport tests exercised

The legacy architecture favours simple module interfaces and local configuration. The v2 design separates core, detection, evaluation, simulation and response packages. Its integration contract also calls for rr/compat.py as the bridge to frozen top-level modules; that file is absent from the supplied archive.

A design distinction is essential: a component's presence establishes that code was supplied, not that it is production-ready or used by the live dashboard. The presentation guide describes a broader rollback-oriented demonstration, but the delivered files do not prove that complete demonstration can be run unchanged.

A shadow snapshot must be taken before destructive modification to preserve original bytes. Taking a snapshot only after an alert may preserve already damaged content. Therefore rollback claims need an integrated pre-change preservation strategy, not merely a passing restore test.

3.2 Entity Relationship (ER) Diagram

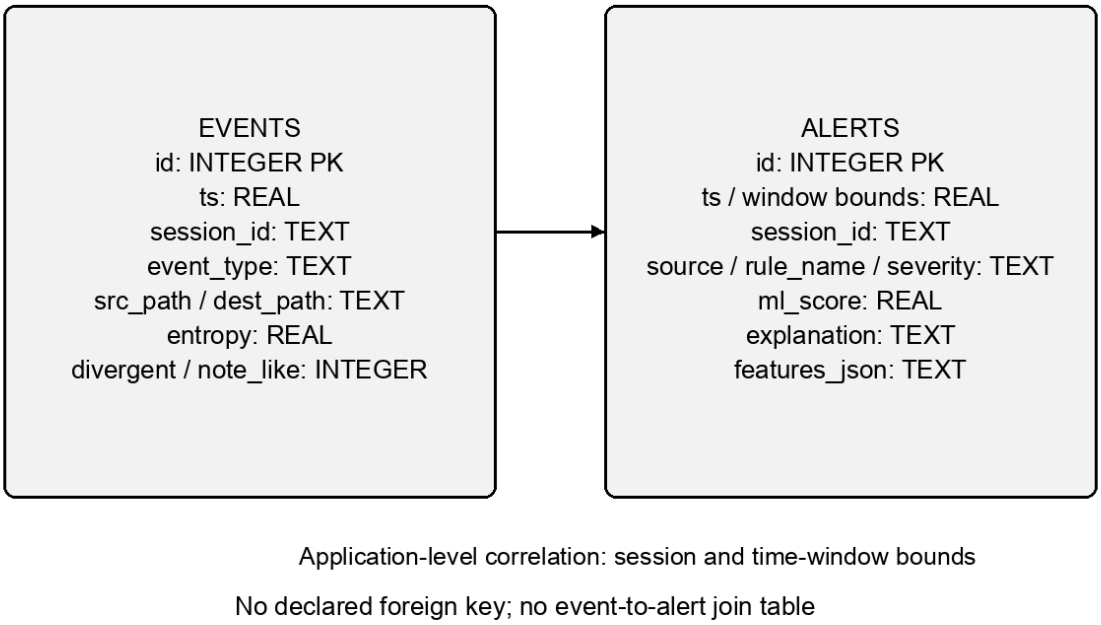


Figure 3.1: Implemented SQLite entities and their logical correlation.

Both tables use independently generated integer primary keys. Alerts carry a session label and time bounds that can be used to retrieve relevant events; the SQLite schema does not declare an event_id foreign key or a session table. This is an application-level temporal association, not an enforced relational constraint.

A future audit-friendly design could add explicit sessions, normalized alert-to-event associations and model-version identifiers. Those additions are not part of the supplied schema. The current features_json column stores a feature dictionary as text, permitting replay of the displayed evidence without introducing a separate feature table.

3.3 Database Tables

Schema below is taken from config.DB_SCHEMA_CREATE and the storage operations in db.py. Epoch timestamps are stored as REAL values, not formatted dates.

events column	Type	Purpose
id	INTEGER PK	Autoincrement event identity
ts	REAL NOT NULL	Observed event timestamp
session_id	TEXT	Experiment/session label; nullable
event_type	TEXT NOT NULL	created, modified, moved or deleted
src_path	TEXT	Original or affected path
dest_path	TEXT	Destination for moved events
entropy	REAL	Sample entropy; nullable if not sampled
divergent	INTEGER	Content/type mismatch flag
note_like	INTEGER	Ransom-note heuristic flag

The events table has indexes on id and ts. Migration logic adds entropy, divergent and note_like to older event tables.

alerts column	Type	Purpose
id	INTEGER PK	Autoincrement alert identity
ts	REAL NOT NULL	Alert timestamp
session_id	TEXT	Session correlation
source	TEXT NOT NULL	rule, ml or fused
rule_name / severity	TEXT	Trigger label and CRITICAL/HIGH/MEDIUM
ml_score	REAL	0–100 display score; not a probability
window_start / window_end	REAL	Time bounds; nullable for direct canary alert
explanation	TEXT	Readable evidence statement
features_json	TEXT	Serialized feature dictionary

The model is a separate joblib artifact. Only load model files from trusted sources because pickle-based model persistence is not safe for untrusted input.

3.4 Snapshots

The snapshot below is generated from read-only queries on the supplied database and the offline component-test log. It is genuine recorded evidence, not a reconstructed Streamlit screenshot. The archive contains no dashboard screenshot, and the Windows dashboard was not executed during this report preparation.

```
ARCHIVED DATABASE EVIDENCE
Source: data/ransomradar.db (read-only query)
Events stored: 302    Alerts stored: 18

SELECT id, source, severity, rule_name FROM alerts LIMIT 2;

ID   SOURCE   SEVERITY   RULE
1    rule     CRITICAL   canary_touched
2    fused    CRITICAL   canary_touched

Offline component test output:
passed=43  failed=0  skipped=5  collect_errors=1

Runner: bundled tools/minipytest.py; not full pytest.
Collection error: missing rr.sim.canaries.
```

Figure 3.2: Archived database counts, sample alert rows and review test output.

The first stored alert is a rule-based CRITICAL canary warning. The next sample row is a fused warning combining the canary rule with learned-profile evidence. These rows illustrate the explanation format; they are not used to compute the experiment CSV's detection rates.

The supplied app.py provides a control panel, event/alert counters, alert table, evidence expander, event-activity plot and recent-event feed. Before submission, a live Windows demonstration can provide additional dashboard images. One source-code issue to check is the sidebar's *canary* filename glob: the configured realistic decoy names do not contain that word, so the displayed count may under-report existing decoys.

4. IMPLEMENTATION

4.1 Implementation Details

The demonstrated application is a set of Python services sharing local SQLite storage. Configuration defines the sandbox, database and model paths. The collector prepares decoy files before starting the observer so their initial creation does not generate detection events. The observer watches the sandbox recursively; a detector thread performs canary checks and window-based evaluation.

Stage	Implementation detail
1. Event collection	RadarHandler receives watchdog callbacks and ignores directories.
2. Noise handling	Configured temporary names are ignored; duplicate same-kind/path events within 0.1 s collapse.
3. Content checks	Modified files are sampled up to 4096 bytes; same-path reads are throttled for 2 s.
4. Feature formation	Timestamp floor-divided by 10 creates fixed, non-overlapping buckets.
5. Decision	Static hits OR a learned-envelope breach create an alert; evidence is fused if both fire.
6. Delivery	SQLite alert persistence, dashboard reading, rate-limited Windows balloon and optional webhook.

Although config.py describes “sliding” windows, the actual legacy feature function groups fixed ten-second buckets. True multi-scale sliding windows are available in the separate rr/core extension. The report uses the implementation behaviour rather than repeating the inconsistent comment.

Laboratory Run Procedure

From a trusted project copy, create a virtual environment, install requirements.txt, then reset only the demonstration sandbox while the collector is stopped. Run python collector.py in one terminal and streamlit run app.py in another. Generate representative benign and bulk activity, run python model.py to train, then restart the collector to ensure it loads the intended model. Run only controlled simulator scenarios. Preserve CSV and database evidence before clearing records.

4.2 Module Description

File / package	Role
config.py	Paths, ten legacy features, thresholds, decoys and SQL schema
collector.py	Watchdog observer, filtering, modified-file sampling and detector thread
forensics.py	Shannon entropy, file signatures, text check and ransom-note heuristics
features.py	Fixed buckets, feature extraction, normal profile and explanation helpers
model.py	Normal-envelope training, Isolation Forest fitting and joblib persistence
rules.py	Static rule hits, fused verdicts, canary checks and live polling
db.py	SQLite initialization/migration, insert/read functions and counts
notify.py	Windows balloon delivery and opt-in webhook
app.py	Streamlit display, evidence inspection, CSV export and clear-record control
simulator.py / demo.py	Controlled workloads and scripted demo of the legacy system
experiment.py	Normal training; scenario capture; four-detector offline evaluation
rr/core + rr/detect	Extended features, overlapping windows, ensemble and conformal logic
rr/respond + rr/notify	Guarded response, shadow journal and push components

Notable archive gaps: rr/compat.py, rr/eval/metrics.py, rr/eval/figures.py, rr/eval/ablation.py, rr/sim/canaries.py, rr/sim/evader.py and rr/sim/benign_workloads.py are not supplied. rr/eval/protocol.py imports rr.eval.metrics, and test_sim_arena.py imports missing simulator components. The gaps must be resolved before claiming full v2 integration.

requirements.txt lists the legacy dependencies but does not list all packages mentioned by the v2 integration contract, such as SciPy and pytest. The rr detector includes an optional PyOD path with fallback logic; dependency behaviour should be verified in a clean environment rather than assumed from the contract.

4.3 Algorithms / Methodology Used

Content Forensics and Feature Extraction

Shannon entropy is $H = -\Sigma\, p(b) \log_2 p(b)$, where p(b) is the frequency of a byte value in the sampled data. Its range is 0–8 bits per byte. High entropy can indicate encrypted content but also occurs in valid compressed or media files. The type-divergence check compares known file extensions against magic-byte prefixes; textual files require at least 85% printable ASCII/whitespace. Non-ASCII legitimate text may therefore be misclassified. [2, 3]

Ten model features are event_rate, files_touched, created, existing_writes, renames, extension_changes, deletions, canary_touched, entropy_max and type_divergence. Ransom-note count is additional rule-only evidence. Existing-write counts exclude paths created in the same bucket. Event rate is bucket event count × 60 / 10.

Static Rules and Learned Normal Envelope

Trigger	Legacy threshold / decision
Canary path	CRITICAL; direct polling path and window rule
Extension changes	At least 3 per bucket → HIGH
Deletions	At least 5 per bucket → HIGH
Event burst	At least 240 events/minute → HIGH
Rename burst	At least 12 renames if event-rate rule did not fire → MEDIUM
Ransom-note heuristic	At least 1 note-like sampled event → HIGH
Learned-envelope breach	Any feature strictly exceeds its stored allowed maximum → ML HIGH

For feature j, envelope_j = max(floor_j, 1.4 × maximum normal-training value_j). The 100-tree Isolation Forest is fitted on the same normal windows, using seed 42; contamination is 0.05 with at least 20 training windows, otherwise 0.10. Its normalized display score corroborates a verdict but does not itself control the legacy ML flag. Thus “ML” in the saved experiment means the full learned envelope, not a pure Isolation Forest classifier. [4]

4.3 Algorithms / Methodology Used (Continued)

Fusion and Explainability

The combined detector is a logical OR, not a voting or weighted-probability scheme. A rule hit and envelope hit together yield source=fused; only rules yield source=rule; only the envelope yields source=ml. The highest triggered rule severity is retained, with ML-only windows labelled HIGH. This improves attack coverage but necessarily preserves any static-rule false positive.

Explanations include the triggered rules, a violated normal-envelope feature, the Isolation Forest display score, descriptive comparisons against normal activity, and sample event descriptions. The label isolation_forest can appear for an ML-only alert even though the actual flag is an envelope violation. The report explains the mechanism explicitly to avoid overstating the classifier's role.

v2 Research Methodology: Present but Not Integrated

The rr detector combines ECOD-style tail evidence with a non-zero Isolation Forest weight, then uses a held-out normal calibration set to obtain conformal p-values. For score s and m calibration scores, $p = (1 + \text{number of calibration scores} \geq s) / (m + 1)$. Sequential decision logic requires sustained low p-values across k windows. These facilities are separate from the legacy experiment CSV.

rr/core adds behavioural ratios, timing, chi-square measurements, entropy aggregates and smoothed features over multiple scales. rr/eval/protocol proposes temporal/session-group splitting and session-level uncertainty to reduce leakage from overlapping windows, but its required metrics import is absent. No v2 detection scores or calibrated false-alarm estimates are claimed in this report.

Detection Delay Method

The archived harness uses the first suspicious event (canary touch, changed extension or deletion), or the first session event as fallback, as its reference. Window alerts are timestamped at bucket end; canary timestamps are the event time for rules/combined. A 0.0 s archived fast-attack delay is therefore a harness convention, not demonstrated zero wall-clock processing time.

5. TESTING

5.1 Testing Strategy

Testing combines static source inspection, analysis of supplied session-level experiment records, read-only inspection of the archived database and fresh execution of available isolated component tests. No live ransomware was used. No untrusted joblib model was deserialized during report preparation. Full Windows runtime, desktop balloons, real webhooks and end-to-end collector operation were not retested here.

Level	Coverage / method	Evidence boundary
Unit/component	ShadowJournal, response policy and push notification tests	Executed using supplied lightweight test runner; fake transports/controllers
Static integration review	Imports, dependencies, live app call path and contract	Can establish missing files and wiring; not runtime throughput
Archived detector evaluation	Same captured sessions scored by four legacy detectors	Supplied CSV, not a newly executed benchmark
Storage validation	Read-only row counts and sample alert contents	Separate saved database; not assumed to match CSV run
Acceptance / Windows	Collector + dashboard + benign and attack demonstrations	Recommended final validation; not performed in this review

Recommended test controls are: isolate a disposable sandbox, reset it only with the observer off, keep normal training separate from evaluation, include large legitimate writes/copies, preserve seeds and package versions, and never allow network or active response during routine unit tests.

The CSV has 32 rows and eight unique sessions, with one row per session-detector pair. There are six synthetic attack sessions and two normal sessions. Twelve delay values are missing because the corresponding detector did not alert; they are excluded from mean detected-delay calculations rather than treated as zero. The data contains no duplicate session-detector pairs.

5.2 Test Cases

ID	Case / expected behaviour	Observed evidence / status
TC01	Empty sample entropy is zero; text and binary distribution differ	Source logic verified; add full pytest assertions before final acceptance
TC02	Canary activity yields CRITICAL warning	Saved alert row shows canary_touched; CSV fast cases detected
TC03	Normal editing should not alert	Archived benign session: all four detector variants no alert
TC04	Legitimate bulk copying should not alert	ML variants no alert; rules and combined false-alarm in archived bulk session
TC05	Fast rewriting/renaming should alert	All variants detect 2/2 archived fast sessions
TC06	Slow rewriting under burst limits should alert	Full ML, entropy-only and combined detect 2/2; rules miss 2/2
TC07	Partial rewrite should not depend on high whole-sample entropy	Full ML and combined detect 2/2; entropy-only and rules miss 2/2
TC08	Snapshot/restore reproduces original bytes	Included ShadowJournal component tests pass under lightweight runner
TC09	Dry-run response must not act on real system	Included response tests pass with fake/no-op controllers
TC10	Notification failure must not crash caller	Included fake-transport push tests pass
TC11	Protected processes/PIDs must be refused	Five parametrized cases skipped by lightweight runner; full pytest required
TC12	v2 simulator tests must collect successfully	Blocked: rr.sim.canaries is absent; collection error

Threshold boundary tests should exercise extension changes of 2/3, deletions of 4/5, event rates of 239/240 and exact envelope equality versus strict exceedance. Also test missing samples, ignored temporary paths, Unicode text, compressed files, midnight/session boundaries and restart/model reload behaviour. These are a validation plan, not additional passed tests.

5.3 Test Results

Table 5.1 summarizes only the supplied experiment_results.csv. “False alarms” counts normal sessions with at least one detection, not the number of alert records or false alarms per hour. Delay means are conditional on detection; missed attacks are not assigned zero delay.

Detector	Attacks detected	Mean detected delay	Normal sessions flagged
Rules only	2/6	0.00 s	1/2
Full learned envelope	6/6	8.23 s	0/2
Entropy-only baseline	4/6	6.12 s	0/2
Combined (OR)	6/6	6.85 s	1/2

Table 5.1: Six attack sessions and two normal sessions; archived legacy evaluation.

Attack type	Rules	Full ML	Entropy only	Combined
Fast	2/2	2/2	2/2	2/2
Stealth	0/2	2/2	2/2	2/2
Partial	0/2	2/2	0/2	2/2

Table 5.2: Detection coverage by attack type (two sessions per type).

The combined method detects all six supplied attack sessions, with a mean archived delay of 6.85 s, but it also flags the legitimate bulk-copy session. Full-envelope detection flags all six attack sessions without flagging either supplied normal session. These tiny synthetic samples do not establish a general 100% detection rate or a guaranteed zero false-alarm rate.

Partial-rewrite sessions have combined delays of 11.4 s and 13.5 s. The entropy-only variant misses both, consistent with the need for evidence beyond entropy. The CSV alone does not identify the exact decisive feature in each session, so attributing every partial detection solely to type divergence would exceed the recorded evidence.

Fresh Component Review Outcome

The archive’s tools/minipytest.py produced 43 passed, 0 failed, 5 skipped and 1 collection error. The skips concern parametrized protected-process/PID cases that this lightweight runner cannot resolve. The simulator test module cannot import missing rr.sim.canaries. This is not a clean full-suite pass and must not be reported as one. Proper pytest and the missing v2 files are required before release.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

RansomRadar demonstrates a practical layered approach to explainable ransomware-like behaviour detection in a controlled folder. Watchdog callbacks, sampled file checks, ten-second feature buckets, static rules and a learned normal envelope produce locally stored warnings with readable evidence. The Streamlit interface provides a convenient way to inspect activity and export records.

The supplied experiments support complementary strengths: direct canary rules catch the fast cases under the harness's timestamp convention, while the full learned envelope covers stealth and partial-rewrite cases that static rules miss. However, the combined OR decision also inherits the bulk-copy false alarm. The synopsis's expected "no false alarms" outcome is therefore not fully achieved in the recorded combined results.

The v2 source adds promising calibrated detection and guarded recovery components, but missing simulation/evaluation files and absent compatibility wiring prevent a claim of complete integration. Fresh component testing is useful supporting evidence, not proof that the Windows application or a whole-machine ransomware defense is ready for deployment.

6.2 Future Scope

• Complete the v2 import graph, compatibility layer and pinned dependency set; run full pytest and a repeatable end-to-end Windows acceptance demonstration.

• Increase evaluation breadth: multiple benign workloads, attack rates and partial-rewrite locations; report files affected before warning, measured wall-clock delay and false alarms per hour.

• Use allowlists and richer content signatures for compressed/media formats, Unicode text and realistic document files. Replace plain-text files carrying Office extensions with valid decoys.

• Investigate USN/ETW/Sysmon collection and process attribution, then test mapping to relevant MITRE ATT&CK techniques such as T1486.

• Integrate pre-change snapshots before claiming rollback of originals. Keep automatic response opt-in, bounded, auditable and reversible, with human gates and protected-process policies.

• Evaluate Linux/macOS collectors and drift-aware recalibration without discarding the passive safety boundary of the approved prototype.

7. REFERENCES

[1] Project team. “RansomRadar: An Intelligent ML Powered Early-Warning System.” Minor Project Synopsis, Computer Science & Engineering, Oriental University, session 2026–27. Supplied PDF.

[2] N. Scaife, H. Carter, P. Traynor and K. R. B. Butler. “CryptoLock (and Drop It): Stopping Ransomware Attacks on User Data.” IEEE International Conference on Distributed Computing Systems (ICDCS), 2016. https://www.cise.ufl.edu/~butler/pubs/icdcs16.pdf

[3] Y. Ineza, G. Jackson, P. Niyonkuru, J. Kevil and A. Serwadda. “Intermittent File Encryption in Ransomware: Measurement, Modeling, and Detection.” arXiv:2510.15133, 2025; revised 2026. https://arxiv.org/abs/2510.15133

[4] scikit-learn developers. “IsolationForest” API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

[5] watchdog contributors. watchdog documentation. https://python-watchdog.readthedocs.io/

[6] Streamlit developers. Streamlit documentation. https://docs.streamlit.io/

[7] SQLite developers. SQLite documentation. https://www.sqlite.org/docs.html

[8] pandas developers. pandas documentation. https://pandas.pydata.org/docs/

[9] Elastic. Detection Rules Explorer. https://elastic.github.io/detection-rules-explorer/

[10] Thinkst. OpenCanary repository. https://github.com/thinkst/opencanary

[11] Project team. RR v2.zip: Python source, README.md, INTEGRATION_CONTRACT.md, RansomRadar_Presentation_Guide.pdf, data/experiment_results.csv, data/ransomradar.db and bundled tests. Supplied archive.

[12] Oriental University. MINOR_PROJECT_REPORT.docx. Supplied college report-format document.

Technical descriptions are grounded in the supplied source files. Tables 5.1–5.2 were recomputed from the archived CSV; Figure 3.2 uses the separate archived database and a fresh lightweight component-test log. No measurements from commercial tools or live malware are asserted.